Система автоматизации проектирования

архитектуры микросервисных систем

AutoSystemDesign

Дичковский Владимир

Цель

Разработать инструмент для автоматизированного построения и проверки архитектурных моделей микросервисной системы на основе функциональных и нефункциональных требований и заданного профиля нагрузки.

Входные данные и результаты

Пользователь передаёт на вход:

- BRD (документ бизнес-требований)
- Функциональные требования
- Нефункциональные требования
- Профиль нагрузки

Пользователь получает на выход:

- C4 Component Diagram
- Sequence-диаграмма на каждое функциональное требование
- OpenAPI 3.1 и Swagger UI

Вход можно задать в YAML или передать документ BRD: требования из документа извлекаются автоматически.

Выходные артефакты

Диаграмма компонентов

- Скомпилированное изображение в выбранном формате (PNG/JPG)
- Исходный код PlantUML
- JSON доменной модели архитектуры компонентов

Диаграмма последовательностей

- Скомпилированное изображение в выбранном формате (PNG/JPG)
- Исходный код PlantUML
- JSON доменной модели диаграммы последовательно-стей

Спецификация API

- Спецификация OpenAPI в формате JSON
- Swagger UI в формате HTML

Изображения предназначены для просмотра, PlantUML — для редактирования диаграмм, JSON — для повторного использования структурированных моделей.

Варианты сервера рендеринга PlantUML

Для компиляции исходного кода PlantUML в изображение доступны три варианта:

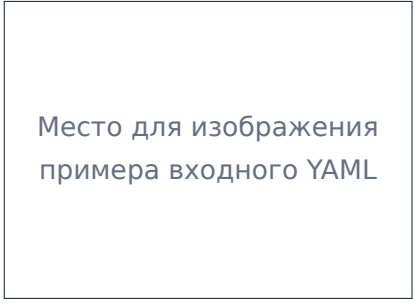
1. **Локальный сервер в Docker Compose**
   Развёртывается вместе с проектом; конфигурация включена в репозиторий.
2. **Официальный сервер PlantUML**
   Используется размещённый разработчиками PlantUML сервер.
3. **Kroki**
   Универсальный сервис рендеринга диаграмм, поддерживающий PlantUML и другие форматы.

Выбор сервера меняет способ рендеринга, но не доменную модель и не исходный код PlantUML.

Пример входного YAML

Структура сценария

```yaml
project_name: "social_network_chat"
description: "Messaging module"
workload:
  baseline_requests_per_second: 500.0
  peak_requests_per_second: 2500.0
  read_ratio: 0.6
requirements:
  - type_identifier: latency
    threshold_value: 150.0
    percentile: 99.0
functional_requirements:
  - identifier: "req_msg_01"
    name: "Direct Message Delivery"
    text: >
      Sender sends a message to a recipient.
```



YAML — один из вариантов входа; требования также извлекаются из документа BRD.

Генерация и проверка

Требования и профиль нагрузки

↓

LLM через LiteLLM

Structured output по классам доменной модели

↓

Типизированные модели и валидаторы

↓

Генерация PlantUML и рендеринг

- LiteLLM позволяет обращаться к разным моделям через OpenAI-совместимый API.
- LLM формирует structured output по классам доменной модели; код PlantUML она не пишет.
- Доменная модель описывает архитектуру, функциональные требования и спецификацию API.
- Рендерер преобразует эту модель в диаграммы; его можно заменить

Модель архитектуры C4

Типы узлов

- Клиент
- Балансировщик нагрузки
- Вычислительный узел
- База данных
- Кэш
- Брокер сообщений

Параметры модели
Число реплик, параллелизм, время обработки, размер очереди, стоимость компонентов и профиль запросов (RPS).

Связи
Разрешены только определённые пары типов узлов. Для каждой связи задаётся протокол взаимодействия.

Реализованные валидаторы диаграммы компонентов

Структура

- Уникальность узлов
- Допустимость связей
- Отсутствие циклов
- Нет изолированных узлов

Семантика и расчёты

- Производитель и потребитель брокера
- Использование кэша и путь при промахе
- Насыщение вычислительных узлов
- Теоретическая задержка и бюджет

Проверки выявляют заданные классы ошибок, но не доказывают корректность всей архитектуры.

Планируемые валидаторы диаграммы компонентов

Планируемые направления

- Симуляция нагрузки с помощью SimPy.
- Модели на основе теории массового обслуживания.
- Анализ очередей, задержек и насыщения под нагрузкой.

Цель: дополнить статические проверки оценкой поведения модели при заданном профиле нагрузки.

Планируемые возможности, не текущая реализация.

Реализованные валидаторы диаграммы последовательностей

Текущие правила

- На каждое функциональное требование — отдельная диаграмма.
- Ссылки на существующие роли и узлы C4.
- Соответствие сценария исходному требованию.
- Использование всех объявленных линий жизни.
- Разные бизнес-роли не используют один клиентский компонент.
- События и уведомления должны быть асинхронными.

Дальнейшее развитие

Расширение проверок семантики сообщений и ветвлений сценария.

REST API: OpenAPI и Swagger

OpenAPI 3.1

- Операции и маршруты
- Параметры и схемы
- Связь с требованиями и узлами C4

Проверки

- Уникальность операций
- Коллизии маршрутов
- Согласованность ссылок



Дальнейшее развитие

- Расширить набор валидаторов архитектуры и сценариев.
- Добавить симуляционные и аналитические проверки нагрузки.
- Сохранить независимость модели от инструмента визуализации.